

Chương 5: Mạng lưới thần kinh nhân tạo

Ph.D. Nguyen Thi Khanh Tien

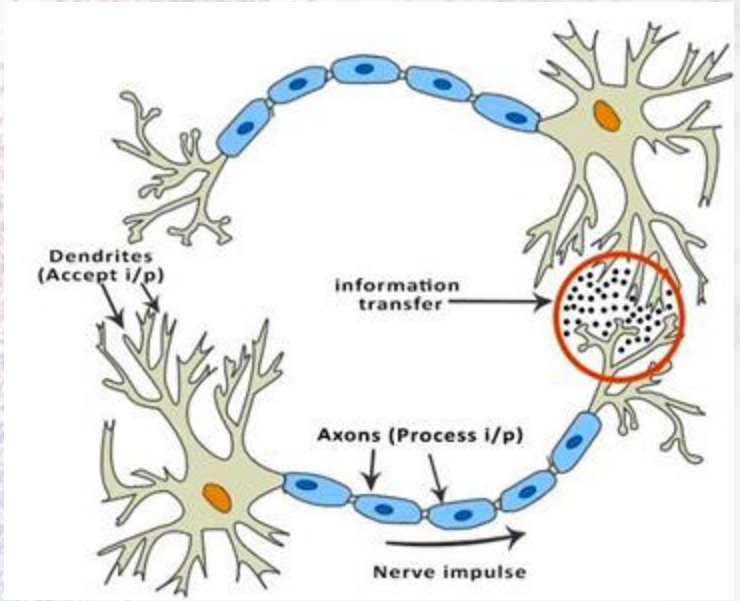
tiennntk@ut.edu.vn

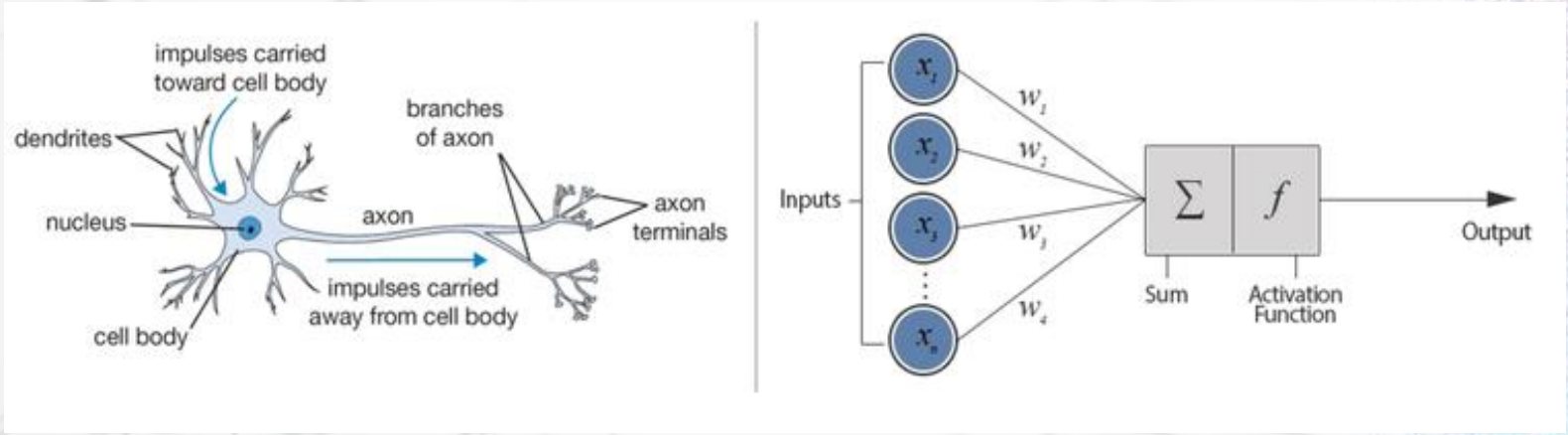
Giới thiệu về Mạng thần kinh nhân tạo (ANN)

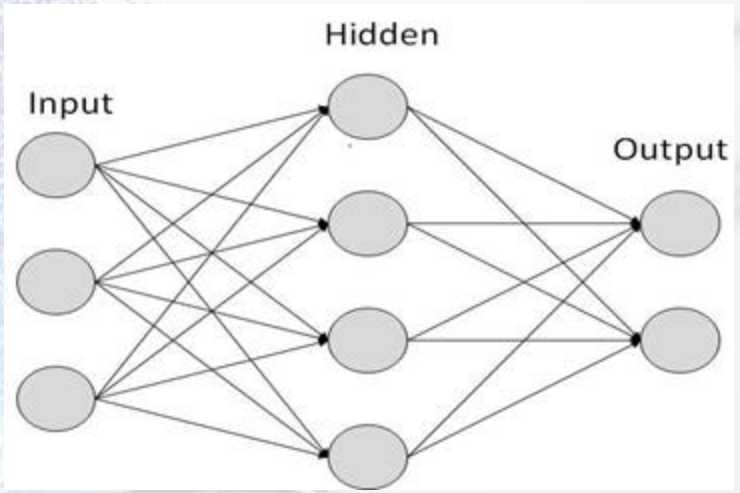
Mạng nơ-ron nhân tạo (ANN) được lấy cảm hứng từ bộ não con người, bao gồm hàng tỷ nơ-ron được kết nối với nhau và nhằm mục đích bắt chước quá trình học tập và ra quyết định của con người bằng cách sử dụng các kết nối và nơ-ron nhân tạo.

- ANN bao gồm nhiều ***nút*** bắt chước các tế bào thần kinh sinh học của não người.

- Đầu ra tại mỗi nút được gọi là ***kích hoạt*** hoặc ***giá trị nút***.
- Mỗi liên kết được liên kết với ***trọng lượng***. ANN có khả năng học hỏi, diễn ra bằng cách thay đổi giá trị trọng số.







A Brief History of Neural Nets

Increase of data transmission speed

1940 ~ 1970: The 1st AI Boom | 1980 ~ 1990: The 2nd Boom | 2006 ~: The 3rd Boom

The advent of the idea of AI



1958: Perceptron

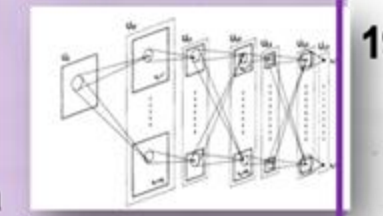
1968: "2001 : A Space Odyssey"



1st AI Winter

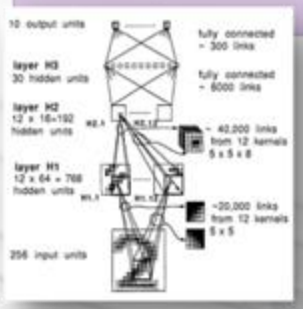
- Limitation of hardware
- Lack of computation algorithms
- Hardships in linearly inseparable data problems

1980: Neocognitron



1986: Backpropagation

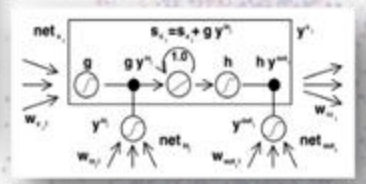
1989: The first practical CNN



2001: Release of Xbox



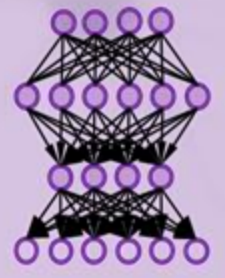
1997: LSTM



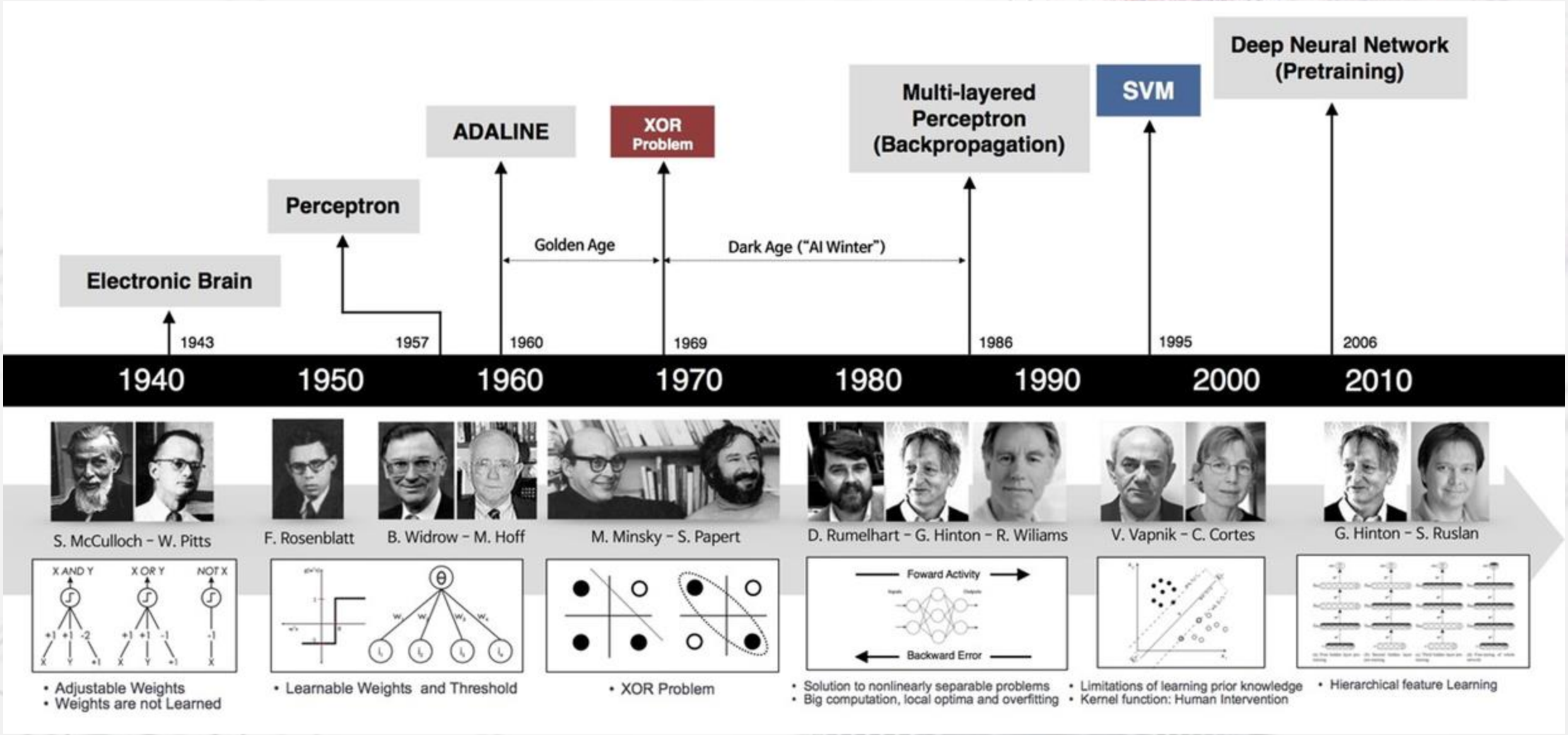
2nd AI Winter

- Limitation of hardware
- Shortage of data sources
- Lack of theories for hyper parameters
- <u>Vanishing / exploding gradient problem</u>

2006: Pretraining of deep belief net



Lịch sử mạng lưới thần kinh



Lịch sử mạng lưới thần kinh

1943: Warren S. McCulloch và Walter Pitts đã xuất bản "Một phép tính logic về những ý tưởng vốn có trong hoạt động thần kinh (liên kết nằm bên ngoài ibm.com)" Nghiên cứu này nhằm tìm hiểu cách bộ não con người có thể tạo ra các mô hình phức tạp thông qua các tế bào não hoặc tế bào thần kinh được kết nối. Một trong những ý tưởng chính nảy sinh từ công trình này là so sánh các nơ-ron có ngưỡng nhị phân với logic Boolean (tức là các câu lệnh 0/1 hoặc đúng/sai).

1958: Frank Rosenblatt được ghi nhận là người có công phát triển perceptron, được ghi lại trong nghiên cứu của ông, "Perceptron: Mô hình xác suất để lưu trữ và tổ chức thông tin trong não" (liên kết nằm bên ngoài ibm.com). Ông đưa công trình của McCulloch và Pitt tiến thêm một bước nữa bằng cách đưa các trọng số vào phương trình. Tận dụng IBM 704, Rosenblatt đã có thể có được một chiếc máy tính để học cách phân biệt các thẻ được đánh dấu bên trái với các thẻ được đánh dấu bên phải.

1974: Trong khi nhiều nhà nghiên cứu đóng góp vào ý tưởng lan truyền ngược, Paul Werbos là người đầu tiên ở Hoa Kỳ lưu ý ứng dụng của nó trong mạng lưới thần kinh trong luận án Tiến sĩ của ông (liên kết nằm bên ngoài ibm.com).

1989: Yann LeCun đã xuất bản một bài báo (liên kết nằm bên ngoài ibm.com) minh họa cách sử dụng các ràng buộc trong lan truyền ngược và sự tích hợp của nó vào kiến trúc mạng thần kinh để huấn luyện các thuật toán. Nghiên cứu này đã tận dụng thành công mạng lưới thần kinh để nhận dạng các chữ số mã zip viết tay do Bưu điện Hoa Kỳ cung cấp.

Mạng thần kinh hoạt động như thế nào?

Các nút xử lý dữ liệu đầu vào thông qua các phép tính toán học, tương tự như cách các nơ-ron sinh học truyền thông tin.

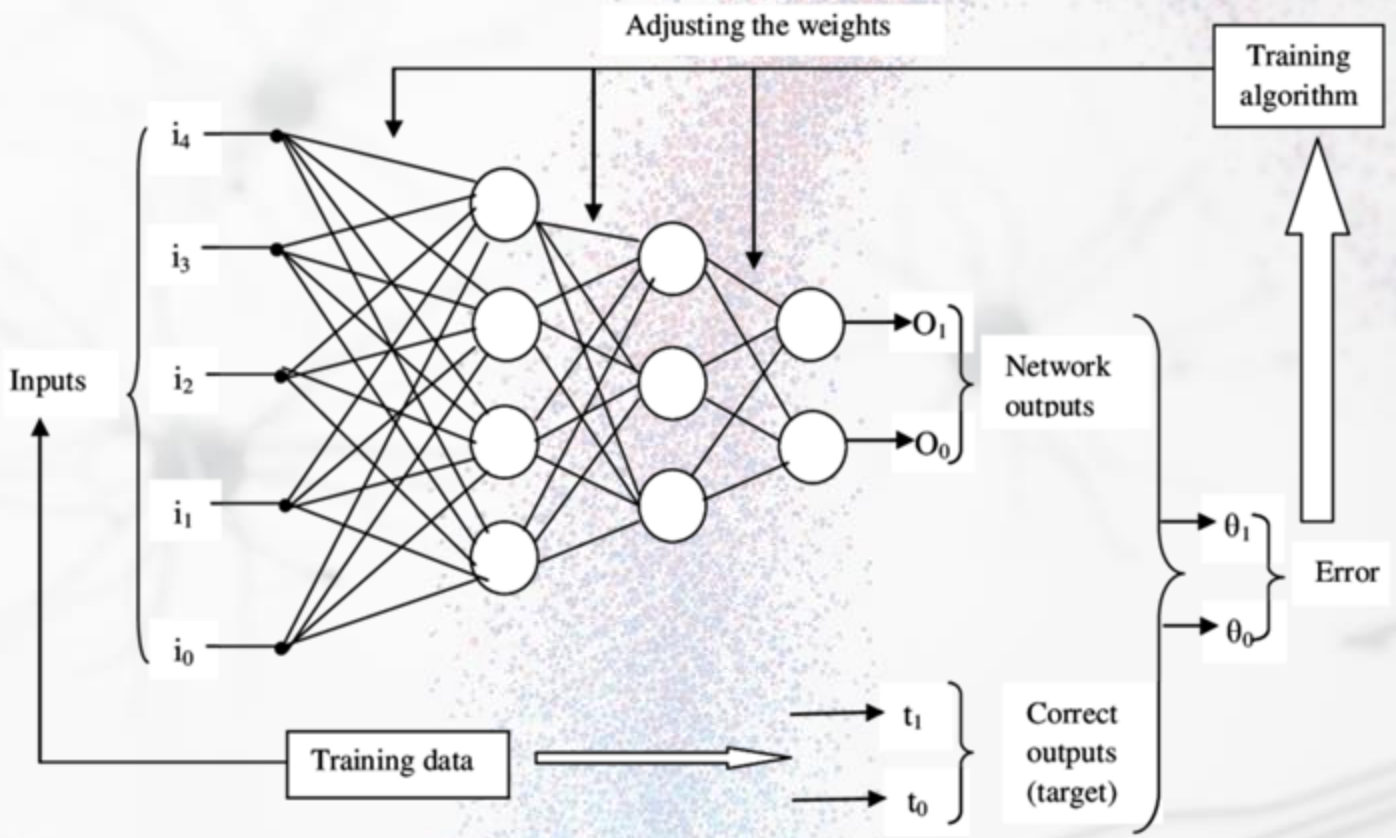
Đây là cách nó hoạt động:

1. **Lớp đầu vào:** dữ liệu ban đầu cho mạng nơ-ron. Đã nhận dữ liệu thô - hình ảnh hoặc văn bản.

2. **Lớp ẩn:** Các lớp này xử lý dữ liệu đầu vào, trích xuất các tính năng và mẫu.

3. **Lớp đầu ra:** Tạo ra kết quả cuối cùng, như phân loại hoặc dự đoán.

Trong quá trình đào tạo, mạng sẽ điều chỉnh các kết nối giữa các nút (trọng số) để giảm thiểu sai sót trong dự đoán của nó. Quá trình này, được gọi là lan truyền ngược, cho phép mạng học hỏi và cải thiện hiệu suất theo thời gian.



Mỗi nút riêng lẻ có mô hình hồi quy tuyến tính riêng, bao gồm dữ liệu đầu vào, trọng số, độ lệch (hoặc ngưỡng) và đầu ra.

$$\sum wixi + bias = w1x1 + w2x2 + w3x3 + bias$$

$$output = f(x) = 1 \text{ if } \sum w1x1 + b >= 0; 0 \text{ if } \sum w1x1 + b < 0$$

Thuật toán lan truyền ngược

lan truyền ngược là một thuật toán cơ bản được sử dụng để huấn luyện mạng lưới thần kinh. Đó là một kỹ thuật học có giám sát giúp điều chỉnh trọng số và độ lệch của mạng để giảm thiểu sự khác biệt giữa đầu ra dự đoán và đầu ra mục tiêu thực tế.

Đây là bản phân tích đơn giản về cách thức hoạt động của nó:

1. Chuyển tiếp:

- Dữ liệu đầu vào được đưa vào mạng lưới thần kinh.
- Dữ liệu được xử lý qua từng lớp, trong đó mỗi nơ-ron thực hiện tổng trọng số của các đầu vào và áp dụng hàm kích hoạt.
- Đầu ra của lớp cuối cùng được so sánh với đầu ra mong muốn và tính toán lỗi.

2. Thẻ lùi:

- Lỗi được truyền ngược qua mạng, từng lớp một.
- Ở mỗi lớp, sai số được sử dụng để tính toán độ dốc của sai số theo trọng số và độ lệch của lớp đó.
- Các gradient này cho biết hướng và độ lớn của trọng số và các điều chỉnh độ lệch cần thiết để giảm sai số.

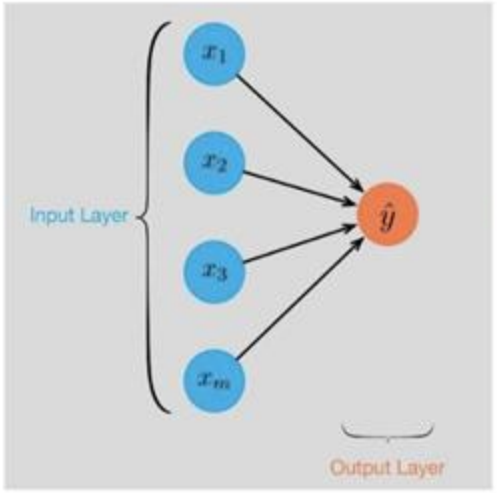
3. Cập nhật trọng lượng và độ lệch:

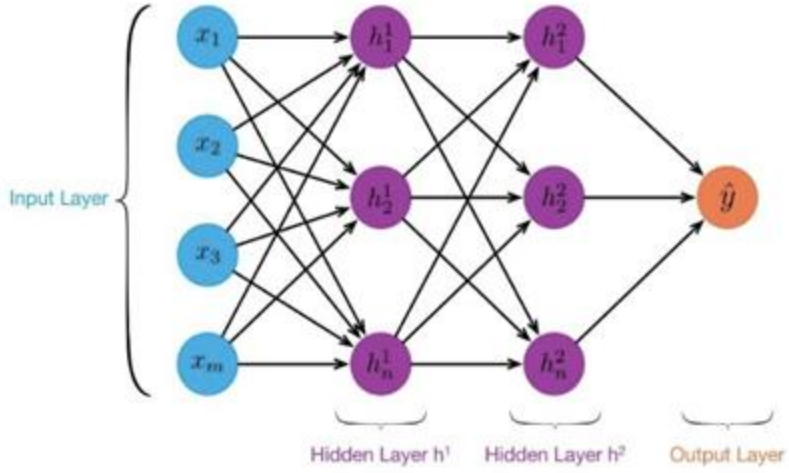
- Các trọng số và độ lệch được cập nhật bằng thuật toán tối ưu hóa như giảm độ dốc.
- Quá trình này được lặp lại nhiều lần, dần dần giảm lỗi và cải thiện độ chính xác của mạng.

Các khái niệm chính:

- **Giảm dần độ dốc:** Một thuật toán tối ưu hóa điều chỉnh lặp đi lặp lại các tham số để giảm thiểu hàm chi phí.
- **Tốc độ học:** Tham số kiểm soát kích thước bước trong quá trình giảm độ dốc.
- **Chức năng kích hoạt:** Một chức năng đưa tính phi tuyến tính vào mạng, cho phép mạng tìm hiểu các mẫu phức tạp.
- **Hàm tổn thất:** Thước đo mức độ hoạt động của mạng, được sử dụng để tính toán lỗi.

Truyền chuyển tiếp (Chuyển tiếp)





First, if we have ***m*** input data (***x1, x2, …, xm***), we call this ***m features***. A feature is just one variable we consider as having an influence to a specific outcome.

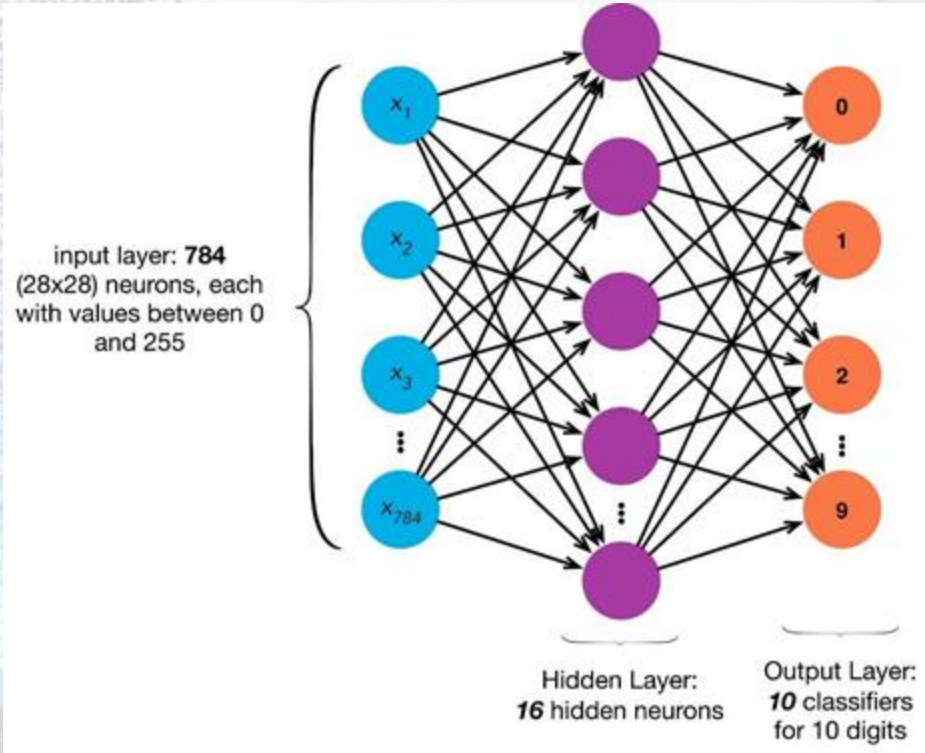
Secondly, when we multiply each of the m features with a weight (***w1, w2, …, wm***) and sum them all together, this is a **dot product**:

$$W \cdot X = w_1x_1 + w_2x_2 + ... + w_mx_m = \sum_{i=1}^{m} w_ix_i$$

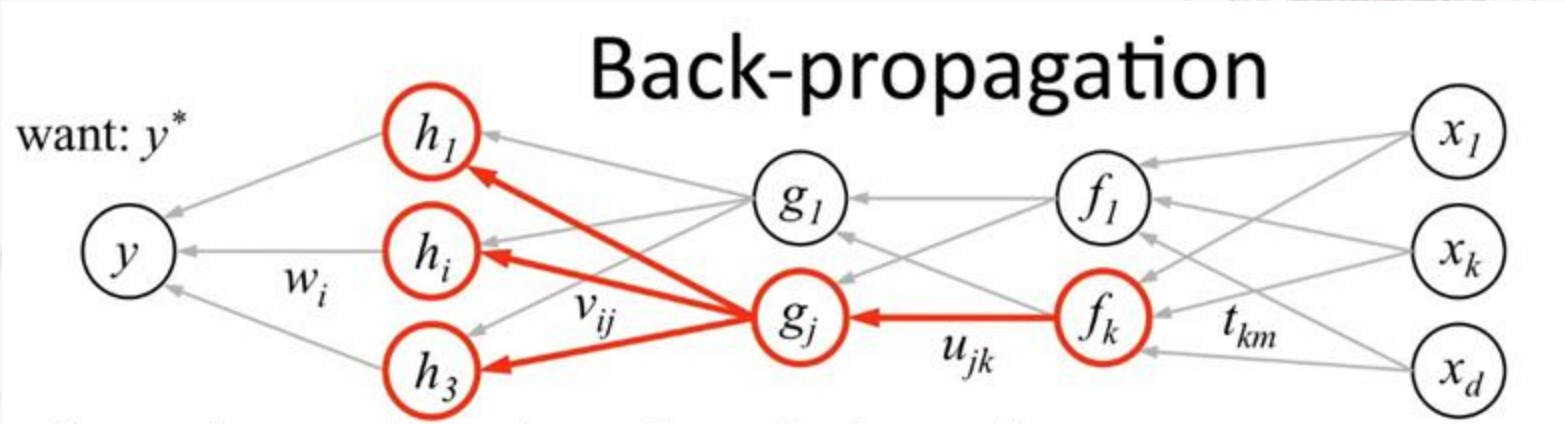
So here are the takeaways for now:

1. With ***m features*** in input ***X,*** you need ***m*** weights to perform a dot product
2. With ***n*** hidden neurons in the hidden layer, you need ***n*** sets of weights (***W1, W2, … Wn***) for performing dot products
3. With 1 hidden layer, you perform ***n*** dot products to get the hidden output ***h:*** (***h1, h2, …, hn***)
4. Then it's just like a single-layer perceptron, we use hidden output ***h:*** (***h1, h2, …, hn***) as input data that has **n features,** perform dot product with 1 set of ***n*** weights (***w1, w2, …, wn***) to get your final output **y_hat**.

Ví dụ về Mạng nơ-ron A với 784 nơ-ron đầu vào, 16 nơ-ron ẩn và 10 nơ-ron đầu ra.



Truyền ngược (Truyền ngược/Truyền ngược)



1. receive new observation $\boldsymbol{x} = [x_1 \ldots x_d]$ and target y^*
2. **feed forward:** for each unit g_j in each layer 1…L
   compute g_j based on units f_k from previous layer: $g_j = \sigma\left(u_{j0} + \sum_k u_{jk} f_k\right)$
3. get prediction y and error $(y\text{-}y^*)$
4. **back-propagate error:** for each unit g_j in each layer L…1

(a) compute error on g_j

$$\frac{\partial E}{\partial g_j} = \sum_i \sigma'(h_i) v_{ij} \frac{\partial E}{\partial h_i}$$

- $\frac{\partial E}{\partial g_j}$: should g_j be higher or lower?
- $\sigma'(h_i) v_{ij}$: how h_i will change as g_j changes
- $\frac{\partial E}{\partial h_i}$: was h_i too high or too low?

(b) for each u_{jk} that affects g_j

(i) compute error on u_{jk}

$$\frac{\partial E}{\partial u_{jk}} = \frac{\partial E}{\partial g_j} \sigma'(g_j) f_k$$

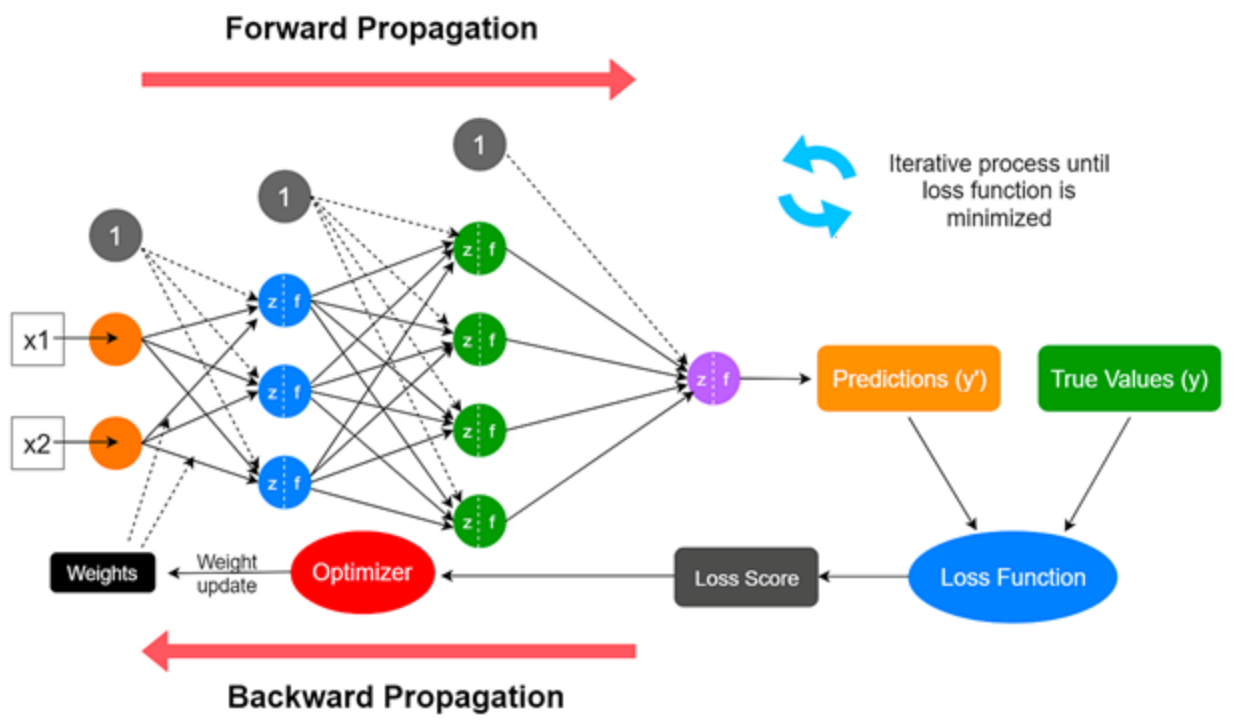
- $\frac{\partial E}{\partial g_j}$: do we want g_j to be higher/lower
- $\sigma'(g_j) f_k$: how g_j will change if u_{jk} is higher/lower

(ii) update the weight

$$u_{jk} \leftarrow u_{jk} - \eta \frac{\partial E}{\partial u_{jk}}$$

https://neptune.ai/blog/backpropagation-algorithm-in-neural-networks-guide

Tính hàm mất mát



3 Chức năng mất khóa

- Hàm tổn thất lỗi bình phương trung bình - Sai số phần trăm tuyệt đối trung bình - Hàm tổn thất Entropy chéo

1. Mean Squared Error Loss Function

Mean squared error (MSE) loss function is the sum of squared differences between the entries in the prediction vector y and the ground truth vector y_hat.

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=0}^{N}(y_i - \hat{y}_i)^2$$

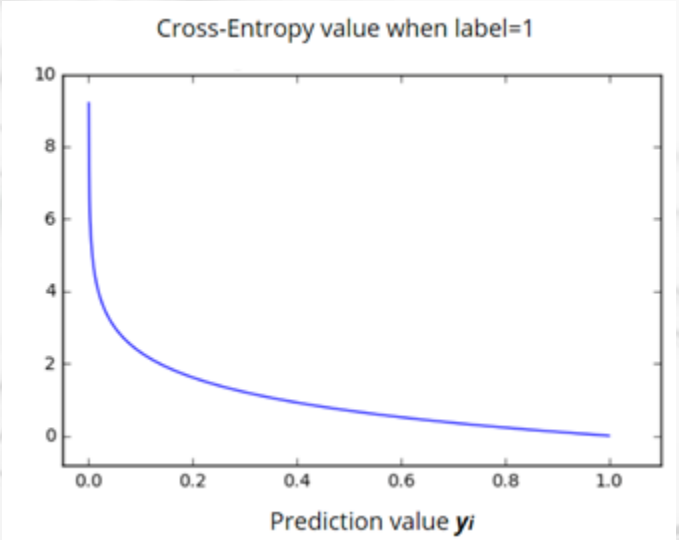
y_i : entries in the prediction vector $\vec{y}$

$\hat{y}_i$: entries in the ground truth label $\vec{\hat{y}}$

2. Mean Absolute Percentage Error

$$MAPE = \frac{100\%}{N}\sum_{i=0}^{N}\frac{|y_i - \hat{y}_i|}{\hat{y}_i}$$

3. Hàm mất Entropy chéo

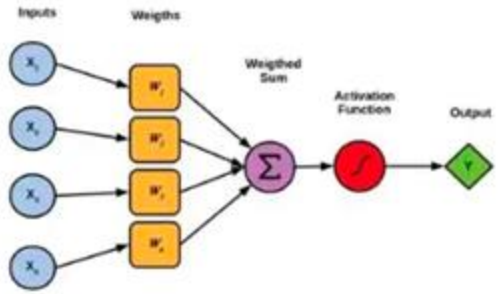


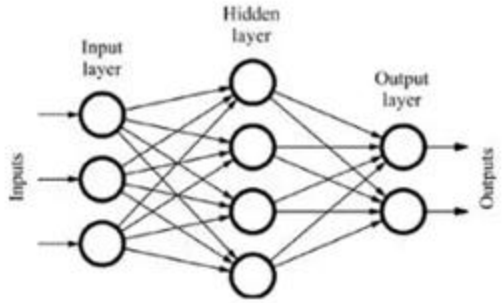
$$\mathcal{L}(\theta) = -\sum_{i=0}^{N}\hat{y}_i \cdot log(y_i)$$

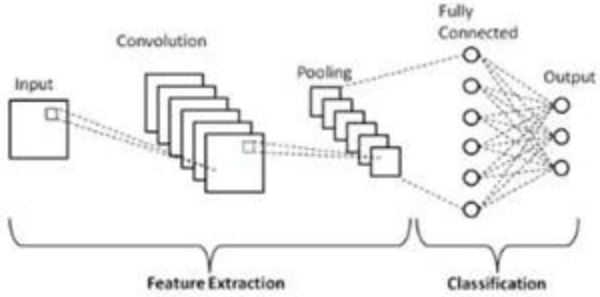
y_i : entries in the prediction vector $\vec{y}$

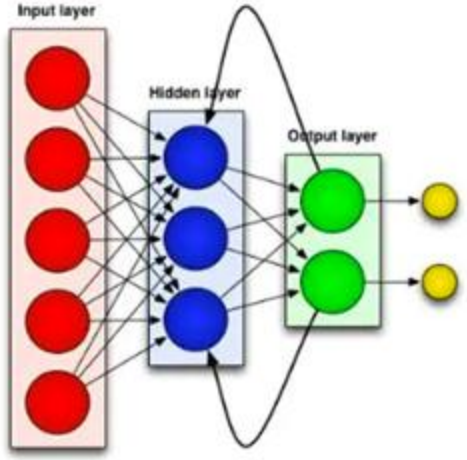
$\hat{y}_i$: entries in the ground truth label $\vec{\hat{y}}$

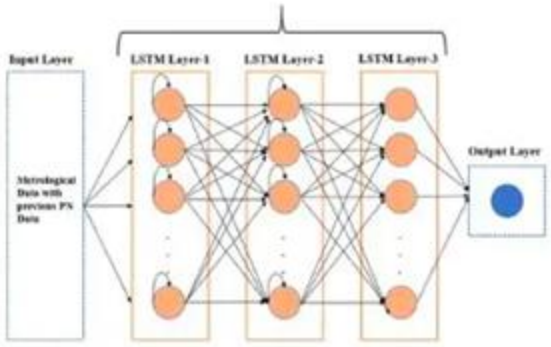
Các loại mạng thần kinh

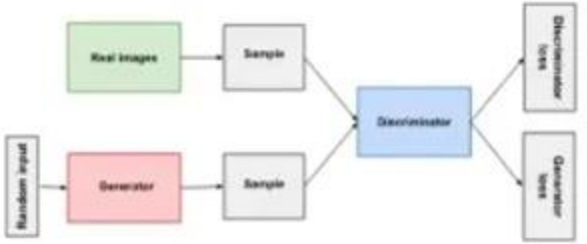












Perceptron

perceptron, một ***khái niệm nền tảng*** trong trí tuệ nhân tạo và học máy, là mô hình đơn giản hóa của nơ-ron sinh học. Đó là một thuật toán đơn giản được sử dụng cho các nhiệm vụ phân loại nhị phân

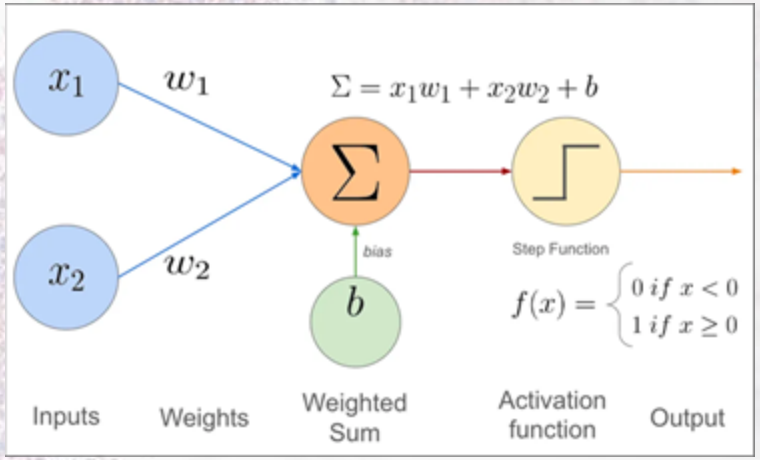
Lược sử. Perceptron được Frank Rosenblatt giới thiệu vào năm 1957. Nó được lấy cảm hứng từ hoạt động của bộ não con người, nơi các tế bào thần kinh xử lý thông tin thông qua các mạng được kết nối với nhau. Rosenblatt đã hình dung ra một cỗ máy có thể học và nhận dạng các mẫu, giống như bộ não con người.

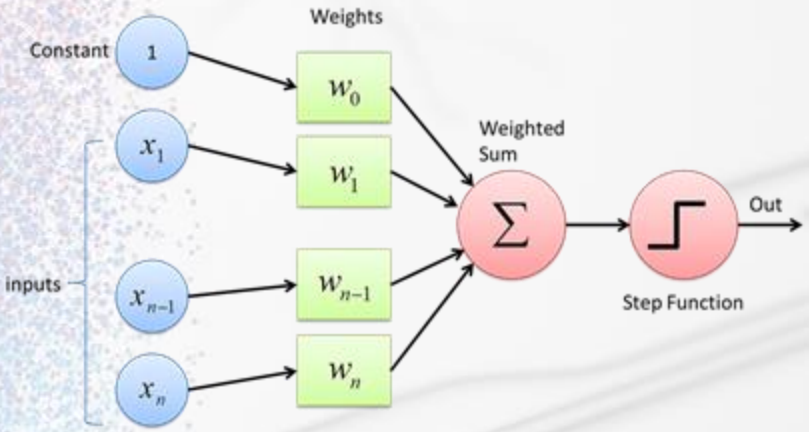
Các thành phần chính của Perceptron:

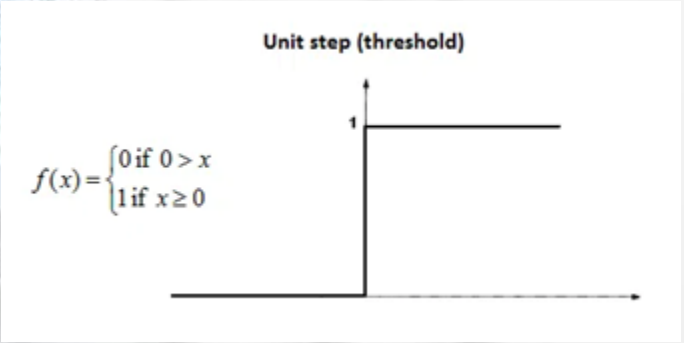
- Đầu vào
- Trọng lượng, Độ lệch. Một số hạng sai lệch được thêm vào tổng có trọng số để điều chỉnh ranh giới quyết định.
- Chức năng kích hoạt. Các hàm kích hoạt phổ biến bao gồm hàm bước, hàm sigmoid và hàm ReLU.

Quy tắc học Perceptron là một thuật toán lặp được sử dụng để điều chỉnh trọng số và độ lệch của perceptron dựa trên sai số giữa đầu ra dự đoán và đầu ra thực tế. Quá trình này tiếp tục cho đến khi perceptron có thể phân loại chính xác dữ liệu huấn luyện.

Những hạn chế của Perceptron. Mặc dù Perceptron là một công cụ có giá trị để hiểu mạng lưới thần kinh nhưng nó cũng có những hạn chế. Nó chỉ có thể phân loại dữ liệu có thể phân tách tuyến tính, nghĩa là nó không thể xử lý các mẫu phức tạp yêu cầu ranh giới quyết định phi tuyến tính. Hạn chế này đã dẫn đến một thời kỳ được gọi là "Mùa đông AI" vào những năm 1970.







Ref:

→https://pub.aimind.so/perceptron-101-the-building-blocks-of-a-neural-network-496f6b9b3826

→https://www.youtube.com/watch?v=-KLnurhX-Pg

Perceptron

Perceptron hoạt động như thế nào

- **Đầu vào** nhận nhiều tín hiệu đầu vào, mỗi tín hiệu đại diện cho một tính năng của dữ liệu.
- **Trọng số và độ lệch.** Mỗi đầu vào được nhân với trọng số tương ứng. Các trọng số này xác định tầm quan trọng của từng đầu vào. Một thuật ngữ sai lệch được thêm vào tổng có trọng số. Độ lệch này cho phép mô hình điều chỉnh ranh giới quyết định.
- **Tổng trọng số.**

  *weighted_sum = (input1 * trọng số1) + (input2 * trọng số2) + ... + thiên vị*
- **Chức năng kích hoạt.** Tổng có trọng số được chuyển qua hàm kích hoạt, hàm này đưa ra tính phi tuyến tính. Một hàm kích hoạt phổ biến là hàm bước: *xuất = 1 if Weighted_sum >= ngưỡng else 0*
  - Nếu tổng có trọng số vượt quá một ngưỡng nhất định thì đầu ra là 1 (lớp dương).
  - Ngược lại, đầu ra là 0 (lớp âm).

Quá trình học tập:

Perceptron học bằng cách điều chỉnh trọng số và độ lệch của nó thông qua một quá trình gọi là huấn luyện. Điều này liên quan đến:

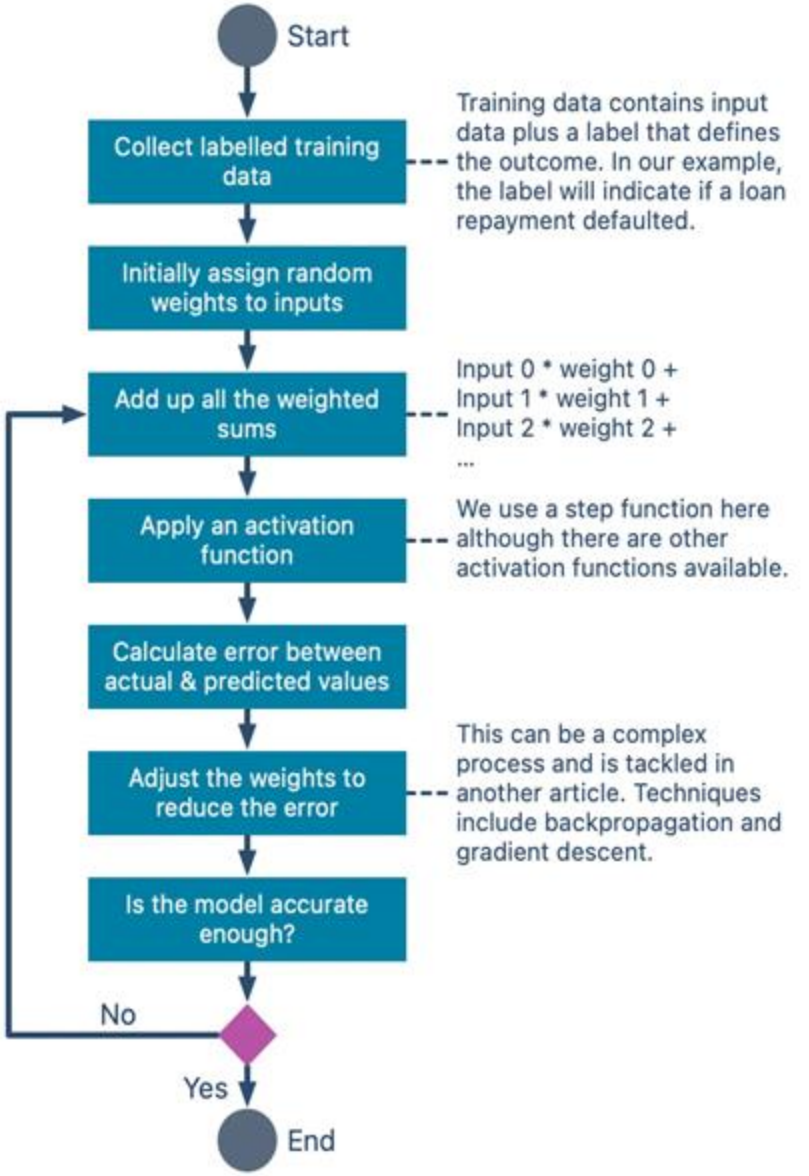
1. Khởi tạo. Trọng số và độ lệch được khởi tạo thành các giá trị ngẫu nhiên. **2. Chuyển tiếp.** Dữ liệu đầu vào được đưa vào perceptron và đầu ra được tính toán. **3. Tính toán lỗi.** Sai số giữa đầu ra dự đoán và đầu ra thực tế được tính toán. **4. Cập nhật cân nặng.** Trọng số và độ lệch được cập nhật bằng quy tắc học, chẳng hạn như quy tắc học của perceptron, để giảm thiểu lỗi:

*trọng lượng_mới = trọng lượng_old + learning_rate * lỗi * đầu vào thiên vị_mới = thiên vị_old + learning_rate * lỗi*

1. Quá trình lặp lại được lặp lại cho nhiều ví dụ huấn luyện, điều chỉnh lặp đi lặp lại trọng số và độ lệch cho đến khi perceptron có thể phân loại dữ liệu một cách chính xác.

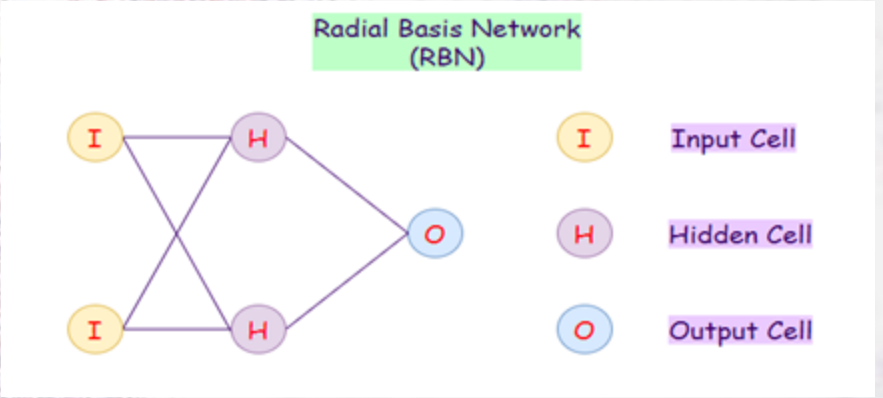
Ví dụ

→https://www.kaggle.com/code/kuzagaya/implementing-a-perceptron-in-python



Mạng chức năng cơ sở xuyên tâm - RBF

Mạng Hàm cơ sở xuyên tâm (RBF) là một loại mạng thần kinh nhân tạo sử dụng các hàm cơ sở xuyên tâm làm hàm kích hoạt. Chúng đặc biệt phù hợp cho các nhiệm vụ xấp xỉ hàm, dự đoán chuỗi thời gian và phân loại.

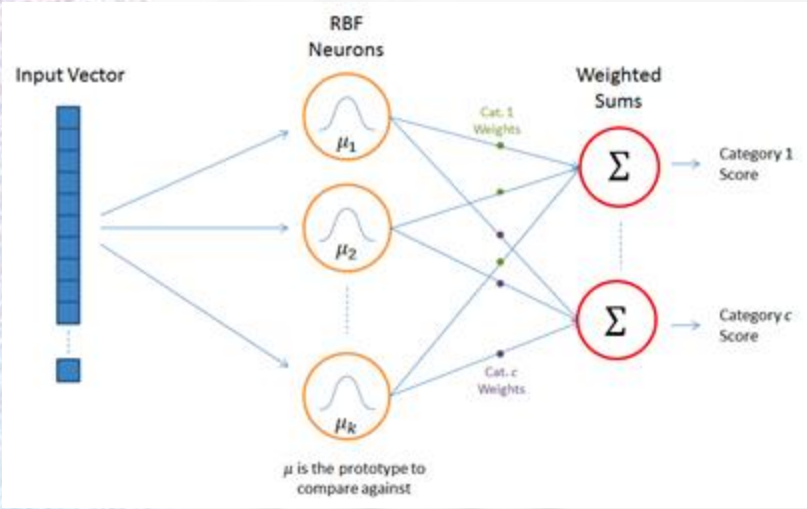


Kiến trúc ba lớp

- Lớp đầu vào nhận dữ liệu đầu vào và chuyển nó đến lớp ẩn.
- Lớp ẩn bao gồm các nơ-ron RBF, mỗi nơ-ron có tâm và chiều rộng riêng. Việc kích hoạt mỗi nơ-ron phụ thuộc vào khoảng cách giữa đầu vào và tâm của nó.
- Lớp đầu ra thực hiện kết hợp tuyến tính các đầu ra của lớp ẩn để tạo ra đầu ra cuối cùng.

Các hàm cơ sở hướng tâm

- Các hàm này được đặc trưng bởi tính đối xứng xuyên tâm, nghĩa là đầu ra của chúng chỉ phụ thuộc vào khoảng cách từ điểm trung tâm.
- CRBF thông thường bao gồm:
  - RBF Gaussian
  - RBF đa bậc
  - RBF đa bậc nghịch đảo
  - Spline tấm mỏng RBF



Quy trình đào tạo

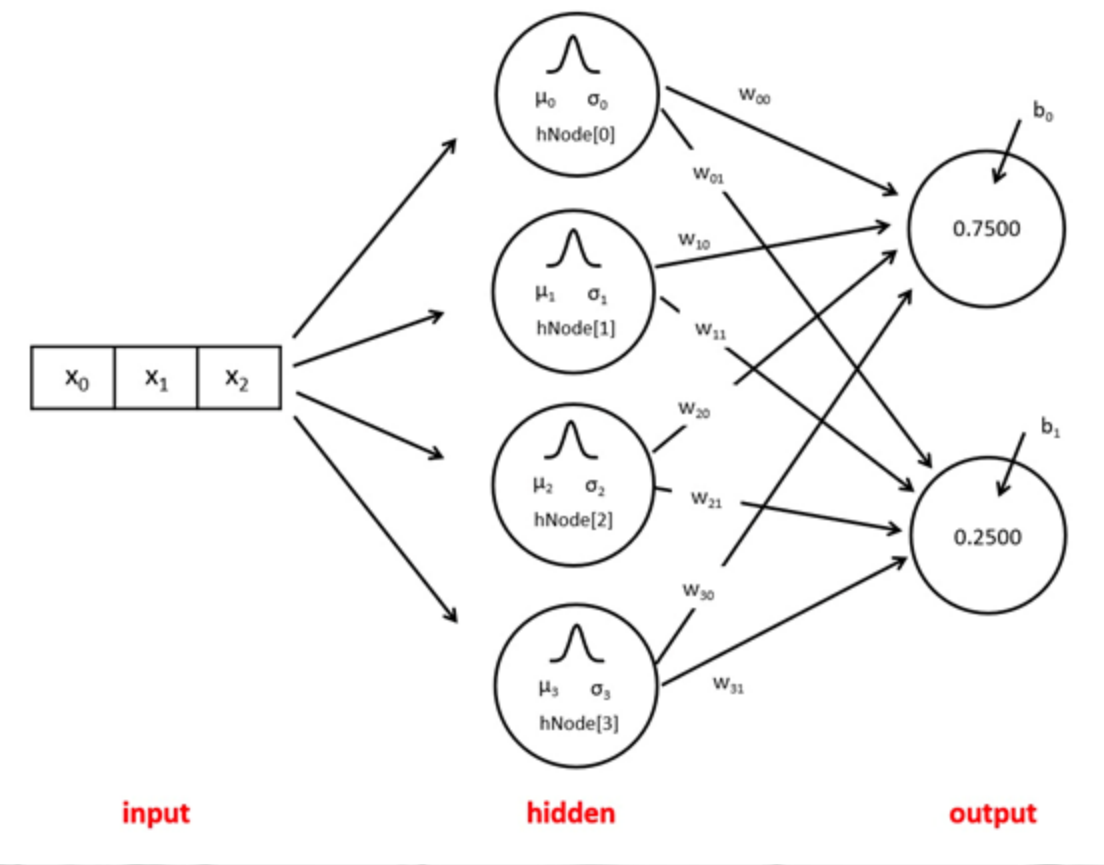
- Xác định trung tâm. Tâm của các nơ-ron RBF có thể được xác định bằng nhiều phương pháp khác nhau, chẳng hạn như thuật toán phân cụm (ví dụ: K-means) hoặc chọn ngẫu nhiên.
- Xác định chiều rộng. Độ rộng của các nơ-ron RBF kiểm soát ảnh hưởng của chúng đến đầu ra. Chúng có thể được cố định hoặc học được trong quá trình đào tạo.
- Tập tạ. Trọng số của các kết nối giữa lớp ẩn và lớp đầu ra thường được huấn luyện bằng phương pháp bình phương tối thiểu tuyến tính.

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Mạng chức năng cơ sở xuyên tâm - RBF

Mạng cơ sở xuyên tâm và lớp Keras tùy chỉnh
→https://kaggle.com/code/residentmario/radial-basis-networks-and-custom-keras-layers

Nội suy cơ sở bán kính từ đầu bằng Python
→https://www.youtube.com/watch?v=Kl9xk9BukNE



Ưu điểm của Mạng RBF:

- **Xấp xỉ phổ quát:** Mạng RBF có thể xấp xỉ bất kỳ hàm liên tục nào với độ chính xác tùy ý.
- **Đào tạo nhanh hơn:** So với các mạng thần kinh khác, mạng RBF thường yêu cầu ít thời gian đào tạo hơn.
- **Hiệu suất khái quát hóa tốt:** Mạng RBF có xu hướng khái quát hóa tốt đối với dữ liệu không nhìn thấy được.

Nhược điểm của Mạng RBF:

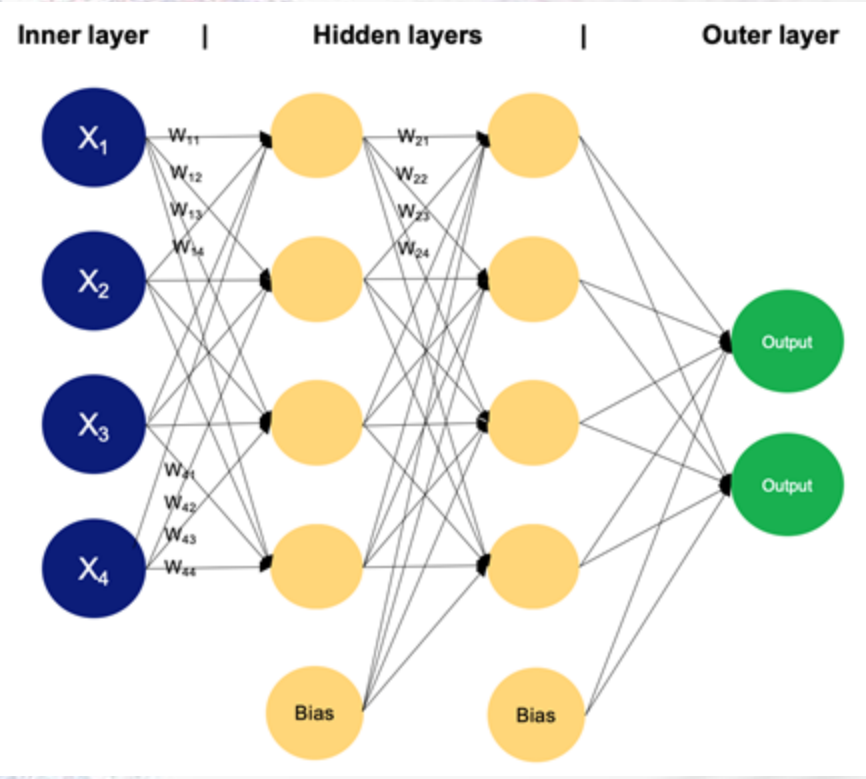
- **Độ nhạy đối với Trung tâm và Chiều rộng:** Hiệu suất của mạng RBF rất nhạy cảm với việc lựa chọn tâm và chiều rộng.
- **Khả năng trang bị quá mức:** Mạng RBF có quá nhiều nơ-ron ẩn có thể phù hợp quá mức với dữ liệu huấn luyện.

Ứng dụng của Mạng RBF:

- **Xấp xỉ hàm:** Mô hình hóa các hàm phức tạp, chẳng hạn như các hệ phi tuyến.
- **Dự đoán chuỗi thời gian:** Dự báo các giá trị trong tương lai dựa trên dữ liệu trong quá khứ.
- **Phân loại:** Phân biệt giữa các loại dữ liệu khác nhau.
- **Hệ thống điều khiển:** Thiết kế bộ điều khiển cho hệ thống động.
- **Nhận dạng mẫu:** Xác định các mẫu trong dữ liệu, chẳng hạn như nhận dạng hình ảnh và giọng nói.

Nhận thức đa lớp (MLP)

MLP là một loại mạng nơ-ron tiếp nối cụ thể bao gồm nhiều lớp nơ-ron được kết nối với nhau. Mỗi nơ-ron trong một lớp được kết nối với mọi nơ-ron ở lớp tiếp theo. Nhiều lớp cho phép mạng trích xuất các tính năng từ dữ liệu đầu vào và dần dần xây dựng biểu diễn các mẫu cơ bản.



Các thành phần chính của MLP:

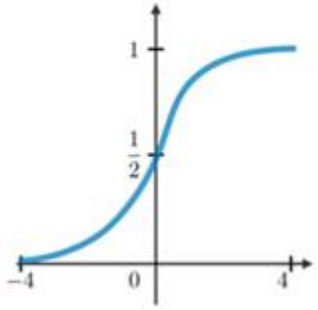
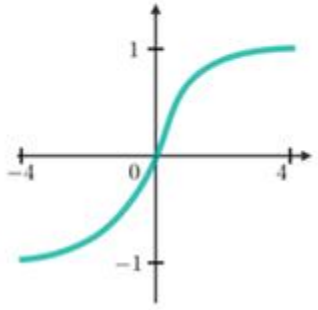
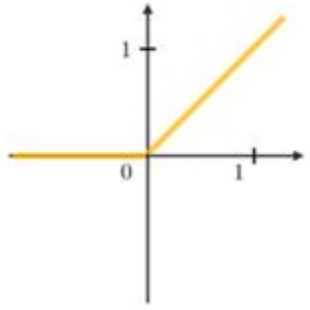
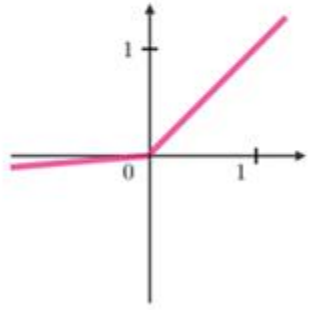
1. **Lớp đầu vào**nhận dữ liệu đầu vào và chuyển nó đến (các) lớp ẩn. 2. **(Các) Lớp ẩn** xử lý dữ liệu đầu vào và chuyển kết quả đến lớp đầu ra. Các lớp này giới thiệu tính phi tuyến tính cho mạng, cho phép mạng tìm hiểu các mẫu phức tạp.

3. **Lớp đầu ra** tạo ra đầu ra cuối cùng của mạng.

Chức năng kích hoạt. Một số chức năng kích hoạt phổ biến được sử dụng trong MLP bao gồm:

- **Sigmoid:** Nén đầu vào thành giá trị từ 0 đến 1.
- **Tanh:** Tương tự như sigmoid, nhưng xuất ra giá trị từ -1 đến 1. - **ReLU**

(Đơn vị tuyến tính chỉnh lưu): Xuất ra giá trị tối đa là 0 và đầu vào.

Sigmoid	Tanh	ReLU	Leaky ReLU
$g(z) = \frac{1}{1+e^{-z}}$	$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$g(z) = \max(0, z)$	$g(z) = \max(\epsilon z, z)$ with $\epsilon \ll 1$
			

$$\text{Weighted Sum} = \sum_{i=1}^{n} (w_i * x_i) + b$$

Nhận thức đa lớp (MLP)

Thuật toán đào tạo cho MLP

Một trong những khía cạnh quan trọng nhất của MLP là quá trình đào tạo, bao gồm việc điều chỉnh trọng số và độ lệch của mạng để giảm thiểu sai số giữa đầu ra dự đoán và đầu ra thực tế.

Thuật toán được sử dụng rộng rãi nhất để huấn luyện MLP là lan truyền ngược.

Truyền ngược

1. Chuyển tiếp:

- Dữ liệu đầu vào được đưa vào mạng.
- Dữ liệu được truyền qua các lớp, với mỗi nơ-ron tính toán hàm kích hoạt của nó.
- Lớp đầu ra tạo ra một dự đoán.

[1]. Thẻ lùi:

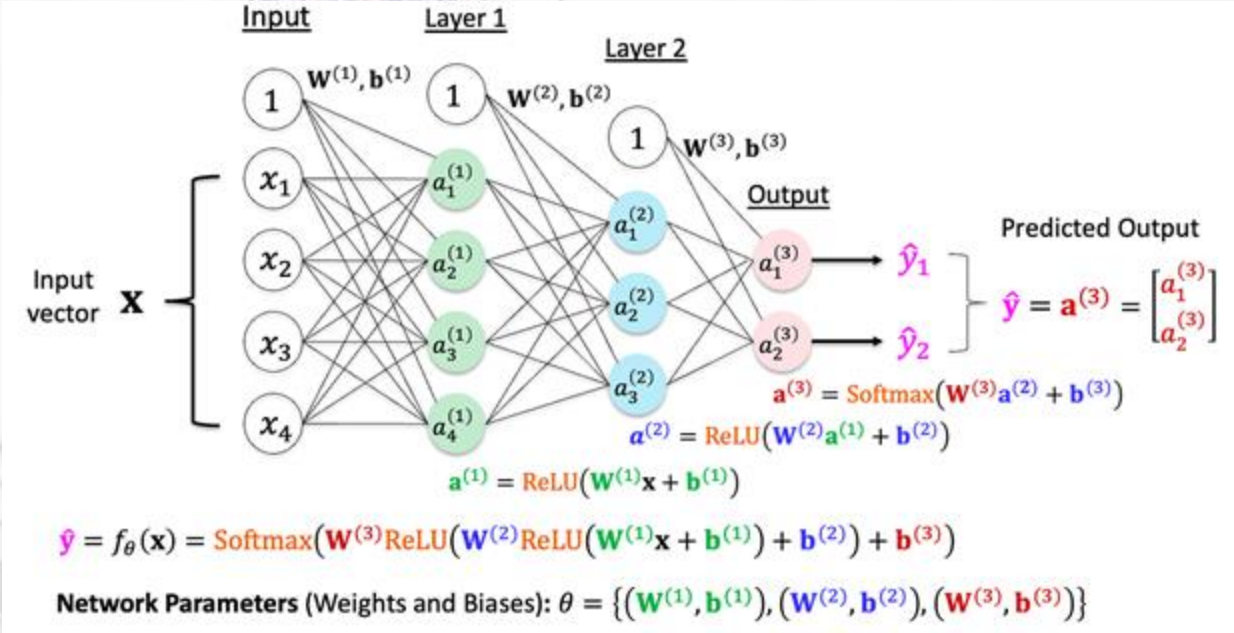
- Sai số giữa đầu ra dự đoán và đầu ra thực tế được tính toán.
- Lỗi này được truyền ngược qua mạng, từng lớp một.
- Trọng số và độ lệch của mỗi nơ-ron được điều chỉnh bằng cách sử dụng phương pháp giảm độ dốc để giảm thiểu sai số.

Ví dụ MLP

→https://github.com/rcassani/mlp-ví dụ/blob/master/mlp_examples.ipynb

Kỹ thuật chính quy hóa. Để giảm thiểu tình trạng trang bị quá mức, các kỹ thuật chính quy hóa có thể được sử dụng:

- **Chính quy hóa L1:** Thêm các giá trị tuyệt đối của trọng số vào hàm mất mát.
- **Chính quy hóa L2 (Giảm trọng lượng):** Thêm các giá trị bình phương của các trọng số vào hàm mất mát.
- **Bỏ học:** Loại bỏ ngẫu nhiên các tế bào thần kinh trong quá trình luyện tập để ngăn chặn sự đồng thích ứng.



References

1) Artificial Intelligence Full Course | Artificial Intelligence Tutorial for Beginners | Edureka
https://www.youtube.com/watch?v=JMUxmLyrhSk

2) Full Machine learning Tutorial
https://www.youtube.com/playlist?list=PLEiEAq2VkUUKiW6VQ0iMjsCpjN2rqG_RO

3) **But what is a neural network?**
→ https://www.youtube.com/watch?v=aircAruvnKk

4) **Gradient descent, how neural networks learn**
→ https://www.youtube.com/watch?v=IHZwWFHWa-w

5) How Computer Vision Applications Work
https://www.youtube.com/watch?v=oGvHtpJMO3M

6) 25 AI Concepts EVERYONE Should Know
https://www.youtube.com/watch?v=0uSyM2DZoOU

7) **Why Neural Networks can learn (almost) anything**

https://www.youtube.com/watch?v=0QczhVg5HaI